PALMPAY CLONE

Complete Code Guide

Every screen, class and flow in this repository, explained in plain terms - with a where-to-edit cheat sheet. Java + XML, MVC-style controllers, immutable models, repository data sources.

Contents

0. Big picture

The app is Java + XML (no Kotlin, no Compose). Each screen has an Activity (the shell) plus a Controller (all behaviour). Data lives in repositories and stores, and screens render immutable model objects built by catalogues - so replacing demo data with a real API later only touches the data classes.

.MainActivity	`Home screen shell: view binding, system bars.`
.ui.HomeScreenController	`All home behaviour and carousels.`
.data.WalletStore	`SharedPreferences: balance, name, key.`
.data.HomeCatalog	`Static home content as models.`
.model.*	`QuickAction, ServiceAction, PromotionCard.`
.profile.*	`ProfileActivity + controller (customisation).`
.transfer.ui.*	`Transfer, Amount, BankPicker screens.`
.transfer.data.*	`Paystack, NUBAN, directory, logos, names.`
.transfer.model.*	`BankInstitution, TransferRecipient, ...`

Layouts live in res/layout (one XML per screen and per row); light/dark values in res/values and res/values-night; XML shapes in res/drawable and PNG art in res/drawable-nodpi.

1. Home screen

MainActivity.onCreate builds its ViewBinding and calls HomeScreenController.bind(), which renders each region in order:

- renderQuickActions() - To Bank / To PalmPay / Savings / Cards; To Bank opens TransferActivity.
- renderServices() - the 8-tile grid; the Data tile carries the orange Promo badge.
- renderPromotions() - Team Up & Save and CashBox cards.
- bindBalanceCard() - WalletStore balance with hide/show eye; Add Money / History are toasts.
- bindHeader() - greeting 'Hi, <name>' from WalletStore; avatar opens Profile.
- bindPromoCarousel() / bindBannerCarousel() - ViewFlippers auto-flipping every 4000 ms with fades; dots synced by a Handler loop.
- bindNavigation() - custom bottom bar (icon masks tinted purple/ink, red dot on Loan, NEW pill on Wealth).

Where to edit

- Tile texts: res/values/strings.xml; which tiles exist: data/HomeCatalog.java.
- Tap behaviour: ui/HomeScreenController.java; carousel speed: the setFlipInterval(4000) calls.
- Colours per theme: res/values/colors.xml + res/values-night/colors.xml.

2. Profile (customisation form)

Three fields and one Save button: Balance -> saveBalance(), Display Name -> saveDisplayName() (drives the home greeting), Paystack API Key -> savePaystackApiKey(). All persisted in the 'palmpay_clone_wallet' SharedPreferences. Home re-reads the balance in onResume().

Where to edit

- Add a field: activity_profile.xml + saveAll() in ProfileScreenController.

3. Transfer to Bank

TransferActivity is the shell (100 ms centre-expand enter animation, result routing). TransferScreenController owns all logic.

3.1 Account input

- A TextWatcher formats digits as '911 241 3798' and calls updateAccountMatch(digits) after every change.
- < 10 digits: history suggestions with progressive purple prefix (SpannableString + ForegroundColorSpan). Any edit clears ALL verification state.
- == 10 digits: the matching-banks request cycle below.

3.2 Matching banks request cycle - showMatchingBanks()

- Clears the container, shows the 'Matching banks' spinner row.
- History banks for this account are added instantly with their saved holder names.
- With a key: PaystackClient.listBanks() (memory -> disk cache -> network) gives the general directory; targets = wallets (opay, palmpay, moniepoint, smartcash, kuda, momo) + up to MAX_VERIFIED_PROBES (10) NUBAN-valid banks.
- One resolveAccount() per target, ALL CONCURRENTLY (OkHttp dispatcher 64 / 40 per host). Successes stream in as rows; failures are remembered in failedBankKeys; the last settle hides the spinner.
- Without a key: offline NUBAN candidates from the GitHub-pages directory, no names.

Policy (user requirement): verification results are NEVER persisted. Deleting and retyping a digit always re-runs the whole request; only the general bank list and logos are cached on device.

3.3 Selecting a bank

- selectMatchedBank()/selectBank(): name in resolvedNames -> instant; known-failed -> red 'Invalid account' banner; otherwise 'Verifying account name' spinner + resolveNameViaPaystack().
- resolveNameViaPaystack(): GET /bank/resolve with account_number & bank_code (snake_case, per Paystack); success -> confirmation row + Next; failure -> banner.
- Next gating: isFormReady() = 10 digits AND bank selected AND a resolved name; a disabled Next is silent.

3.4 Support classes

NubanBankResolver	`Static CBN check-digit math (3-7-3 weights, mod 10).`
BankNameNormalizer	`Suffix stripping, aliases, dedupe by code/canonical name.`
BankLogoResolver	`Canonical name -> bundled rounded logo resource.`
BankLogoLoader	`Downloads logos, circle-clips ONCE, memory + disk cache.`
BankDirectoryRepository	`Banks.json fetch + offline fallback, deduped.`
PaystackClient	`OkHttp; cached general list; resolve; no result persistence.`

4. Bank picker

- 'Frequently Used Bank' grid: verified NUBAN matches for the passed account top it; otherwise a fixed set.
- Alphabetical full list from the directory with section headers and an A-Z rail; search filters by name/code.
- Logos via BankLogoLoader; selection returns to selectBank().

5. Amount screen

- Custom keypad (GridLayout amount_keypad); amount field suppresses the system keyboard; Note field shows Gboard and hides the keypad (focus choreography); manifest uses adjustResize.
- Live comma grouping (formatAmount); parseAmount strips commas.
- Validation 10.00-200,000.00 with red amount_error and disabled Next; >= 10,000 shows the FIRS stamp-duty notice (13sp).
- Balance line 'Balance: N 0.00 CashBox: <wallet>' (Balance is always 0 by product rule).

Where to edit

- Rules: AmountScreenController constants MIN_AMOUNT / MAX_AMOUNT / STAMP_DUTY_THRESHOLD; keys in activity_amount.xml.

6. Persistence policy

wallet prefs	`available_balance, display_name, paystack_api_key`
paystack prefs	`bank_directory_json (GENERAL list only)`
cache dir	`bank_logos/ processed rounded logos`
never stored	`resolved names, failed banks, matching lists`

7. Themes

res/values and res/values-night define the same colour names and Android picks per system theme; drawable-night/ overrides art. Screens only reference @color/..., so a theme change is a single value edit.

8. Tests & CI

- Catalogue/model unit tests guard invariants; NubanBankResolverTest uses the CBN circular's own examples.
- BottomNavigationLabelTest (Robolectric) inflates MainActivity and asserts the five labels render.
- PaystackLiveProbeTest is an @Ignore-d live diagnostic.
- CI (.github/workflows/android.yml, user-supplied): unit tests, assembleDebug, APK upload on every push.

9. Cheat sheet - where do I change...?

Name / balance / key storage	`data/WalletStore.java`
Home tiles & icons	`data/HomeCatalog.java`
Home behaviour / carousels	`ui/HomeScreenController.java`
Bottom bar look	`layout/bottom_nav_item.xml + bindNavigation()`
Transfer flow	`transfer/ui/TransferScreenController.java`
Matching-bank policy	`showMatchingBanks(), WALLET_PROVIDERS, MAX_VERIFIED_PROBES`
Name resolution HTTP	`transfer/data/PaystackClient.java`
Bank math	`transfer/data/NubanBankResolver.java`
Name variants / dupes	`transfer/data/BankNameNormalizer.java`
Logos	`BankLogoLoader/Resolver + drawable-nodpi/`
Amount rules	`transfer/ui/AmountScreenController.java`
Picker	`transfer/ui/BankPickerScreenController.java`
Any text	`res/values/strings.xml`
Any colour	`res/values/colors.xml (+ values-night)`
Any shape / rounded bg	`res/drawable/*.xml`